

第7章 排序

建议学时:4-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 掌握七种排序的算法思想与实现,理解**稳定性**并会给出反例
- 能用求和记号 Σ 推导插入排序的最坏 $O(n^2)$ 与"近有序 $O(n)$ "性质
- 掌握快排的**三数中值分割法**与小小区间插入优化,说清最坏 $O(n^2)$ 何时发生
- 能用决策树直觉解释**比较下界** $\Omega(n \log n)$,理解桶排为何能突破
- 完成从 C 的 `qsort` 到 Python `sorted(key=)` 的迁移,能按数据特征选型

💡 从 C 到 Python:本章主题如何迁移

排序是工程中出现频率最高的操作:排行榜、去重、二分查找前置、Top-K.....本章把第2章的复杂度分析、第6章的二叉堆全部用上。路线:经典排序(\$7.1-\$7.4) → 快排(\$7.5) → 桶排与外部排序(\$7.6) → 总表选型(\$7.7)。请务必亲手写一遍每一种排序——它们最常被要求默写。

在 C 里,通用排序武器只有 `qsort`,三个痛点——① `void*` 丢类型;② 比较语义是"负/零/正"三值;③ 只能整体排序。Python 的 `sorted` 用 `key=` 参数逐一解决:键函数推导类型、`functools.cmp_to_key` 桥接三值比较、任意可迭代对象随意截取;它内部是"归并 + 插入"混合体(Timsort),稳定且自适应,本章的小区间插入、利用近有序等优化全内置。

📦 前置知识自查

数学前置:求和记号 Σ 与 $\Sigma_{i=1}^n i = n(n+1)/2$;递推式 $T(n) = 2T(n/2) + n$ (第2章)。

Python 前置:列表、`range` 与 `for`;切片 `a[i:j]` 与拼接; `functools.cmp_to_key` (\$7.5 对照)。

依赖章节:第2章复杂度;第6章二叉堆(堆排序复用**下滤**与**下滤建堆**,注意 1 基/0 基差异)。

🚀 本章学习路线

1. **第一阶段(1 小时):**读 \$7.1-\$7.2,默写插入与希尔排序
2. **第二阶段(1.5 小时):**读 \$7.3-\$7.4,手写堆排与 merge
3. **第三阶段(1.5 小时):**读 \$7.5-\$7.6,吃透三数中值与分割
4. **第四阶段(实验,1 小时):**完成章末实验。验收:能默写全部经典排序,解释快排何时最坏

本章知识导图

- §7.1 插入排序:扑克牌直觉、 Σ 推导、近有序 $O(n)$ 、稳定性
- §7.2 希尔排序:希尔增量与 Hibbard、约 $O(n^{3/2})$ 、不稳定
- §7.3 堆排序:复用第6章的二叉堆(朴素版、原地版、ADT 关系)
- §7.4 归并排序: $T(n)=2T(n/2)+n$ 、merge、稳定与 $O(n)$ 空间
- §7.5 快速排序:三数中值、分割、小区间插入、qsort $\rightarrow$ sorted(key=) 对照
- §7.6 桶式排序与外部排序:桶排 $O(n)$ 、比较下界 $\Omega(n \log n)$ 、多路归并
- §7.7 排序算法总表:七种排序大表、选型决策树

§7.1 插入排序:算法、复杂度与稳定性

7.1.1 扑克牌直觉与算法步骤

直觉一句话:像整理扑克牌——每摸一张新牌,插到已排好序的牌中的正确位置。算法把数组分成"已排序前缀 $a[0..i-1]$ "与"未排序后缀 $a[i..n-1]$ ":第 i 步用 key 暂存 $a[i]$,从右往左找位置,挡路元素右移一格,落位。

示例 7-1 插入排序(key 暂存与右移)

```
def insertion_sort(a, left=0, right=None):    # 对 a[left..right] 升序排序
    if right is None:
        right = len(a) - 1
    for i in range(left + 1, right + 1):
        key = a[i]
        j = i - 1
        while j >= left and a[j] > key:      # 严格大于才右移:稳定
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
```

7.1.2 复杂度推导:把循环翻译成求和

最坏情形(完全逆序):第 i 个元素要与前面 i 个元素逐一比较并右移,总比较次数 $= \sum_{i=1}^{n-1} i = n(n-1)/2 = O(n^2)$ 。平均

情形:第 i 个元素平均只比较 $i/2$ 次,总次数 $\approx n(n-1)/4$,仍 $O(n^2)$ 。最好情形(已有序):每个元素只比较 1 次, $O(n)$ 。

更本质的刻画是**逆序对(inversion)**:满足 $i < j$ 且 $a[i] > a[j]$ 的数对。插入排序每次右移恰好消除一个逆序对,比较/移动次数 = 逆序对数——逆序对很少时排序就快,"近有序 $O(n)$ "的来源。

7.1.3 近有序数组:接近 $O(n)$ 的性质

若数组"大体有序"(逆序对只有 $O(n)$ 个),插入排序只需 $O(n)$ 次比较与移动,接近线性——章末实验将实测。但也暴露致命局限:每次只交换相邻元素,跨越 k 个位置要走 k 步。例:[2, 3, 4, 5, 6, 7, 8, 1] 中元素 1 走到开头要走 7 步。"让元素一次走远"正是 §7.2 希尔排序的动机。

7.1.4 稳定性:相等元素保持相对次序

稳定性(stable):值相等的元素保持原相对次序。插入排序是稳定的——右移条件写"严格大于" $a[j] > key$,相等立即停止。为何重要?先按姓名排、再按分数排,若第二次稳定,同分者仍按姓名有序——多关键字排序可逐关键字轮流排。

⚠ 误区 1:内层循环里不暂存 key,直接与 a[i] 比较

新手常写 `while j >= left and a[j] > a[i]:` —— `a[i]` 在右移中可能已被覆盖, key 丢失。正确姿势:先 `key = a[i]`, 循环里只移动 `a[j]`。

§7.2 希尔排序:增量序列与远距离搬移

7.2.1 动机:让元素一次走远

插入排序慢在“每次只走一格”。希尔排序:先用大步长“预排序”让元素大体到位,再逐步缩小步长,步长 1 时就是普通插入排序。步长叫**增量(gap)**:把下标相差 h 的元素看作同一组,对每组做插入排序——元素一次跨越 h 个位置。

7.2.2 增量序列:希尔增量与 Hibbard

- **希尔增量**: $n/2, n/4, \dots, 1$ (每次折半)。实现最简单,但增量互为倍数,奇偶下标各自成组,最坏退化为 $O(n^2)$;
- **Hibbard 增量**: $1, 3, 7, 15, \dots, (2^k - 1)$ 。相邻增量互质,避开坏结构,最坏 $O(n^{3/2})$ 。

示例 7-2 希尔排序(希尔增量版)

```
def shell_sort(a):
    n = len(a)
    gap = n // 2
    while gap > 0:
        # 希尔增量: n/2, n/4, ..., 1
        for i in range(gap, n):
            # 对每组做插入排序
            key = a[i]
            j = i
            while j >= gap and a[j - gap] > key:  # 一次跳 gap 格
                a[j] = a[j - gap]
                j -= gap
            a[j] = key
        gap //= 2
```

7.2.3 复杂度与稳定性

希尔排序的精确复杂度至今未完全解决。工程口径(与速查表一致):希尔增量最坏 $O(n^2)$, Hibbard 增量最坏 $O(n^{3/2})$, 平均都约 $O(n^{3/2})$ 。它不稳定:大间隔插入把相等元素“隔空换位”。反例: $[5_a, 5_b, 1]$, $\text{gap} = 2$ 时 $(5_a, 1)$ 排成 $(1, 5_a)$, 数组变 $[1, 5_b, 5_a]$, 最后 $\text{gap} = 1$ 也救不回来。

⚠ 误区 2:增量序列忘了最后是 1,或增量间互为倍数

① 增量必须以 1 收尾,否则只保证“每组内有序”,整体未必有序——`gap` 写成 `gap > 1` 就错。② 希尔增量互为倍数,偶数下标元素永远只在偶数组内比较,是最坏 $O(n^2)$ 的根源;改用 Hibbard $(2^k - 1)$ 互质序列即可避开。

§7.3 堆排序:复用第6章的二叉堆

7.3.1 朴素版:建堆 + 连续删除最小

第6章结尾预告过:堆排序 = 二叉堆的“全量消费”。朴素版两步:① 下滤建堆 $O(n)$;② 连续 `deleteMin` n 次,每次 $O(\log n)$ 。总代价 $O(n \log n)$,但要一块 $O(n)$ 输出数组。

7.3.2 原地版:最大堆 + 交换 + 缩小

原地版把输出数组省掉:改用**最大堆**。建最大堆后根即全局最大;把根与末尾交换,堆大小减 1,再下滤新根——被换到末尾的最大元素就此“钉死”在最终位置。反复 $n-1$ 次,全程只用原数组。

注意下标习惯:原地堆排用 0 基(左子 $2i+1$ 、右子 $2i+2$ 、父 $(i-1)//2$ 、最后非叶 $n//2-1$),与第6章手写堆的 1 基不同。

示例 7-3 原地堆排序(0 基下滤)

```
def sift_down(a, i, n):          # 下滤 a[i]; n 为当前堆大小
    while True:
        largest = i
        for c in (2 * i + 1, 2 * i + 2):    # 左子、右子(0 基)
            if c < n and a[c] > a[largest]:
                largest = c
        if largest == i:
            break
        a[i], a[largest] = a[largest], a[i]
        i = largest

def heap_sort(a):
    n = len(a)
    for i in range(n // 2 - 1, -1, -1):    # 1. 下滤建堆  $O(n)$ 
        sift_down(a, i, n)
    for i in range(n - 1, 0, -1):          # 2. 逐个钉死最大元
        a[0], a[i] = a[i], a[0]
        sift_down(a, 0, i)
```

7.3.3 复杂度、稳定性与 ADT 关系

建堆 $O(n)$ + $n-1$ 次下滤 $\times O(\log n) = O(n \log n)$, 最好最坏都是它,且**原地**只要 $O(1)$ 额外空间。它不稳定:反例 $[4_a, 4_b, 3]$ 建堆后根 4_a 与末尾 3 交换得 $[3, 4_b, 4_a]$, 4_b 最终排在 4_a 前。

从 ADT 视角看:优先队列承诺“每次取出最小元”,堆排序等于把“取最小”执行 n 次,顺便得到完整有序序列。“边插入边取最大值”用优先队列,“一次性排好序”用排序 (§7.7)。

C 的手写堆排序	Python 的标准库
手写下滤、建堆、交换,约 30 行	<code>heapq.heapify</code> + 连续 <code>heappop</code> 两行

⚠ 误区 3:堆排序把 1 基公式抄进 0 基代码

第6章手写堆用 1 基,原地堆排用 0 基。把 1 基的“左子 $2i$ 、父 $i/2$ ”抄进来会访问错位置,堆序被悄悄破坏、输出“看起来差不多但不对”。对策:下标公式写进注释,写完用“验证堆序”断言跑一遍。

§7.4 归并排序:分治的教科书

7.4.1 分治三步与递推式

归并排序是分治思想的教科书案例:① 分割:数组从中间切成两半;② 递归:各自排序;③ 合并:两个有序半区线性合并成一个有序整区。递推式即第2章的 $T(n) = 2T(n/2) + n$,解得 $O(n \log n)$ 。

7.4.2 merge:两个有序序列的线性合并

合并直觉: 像两队人按身高排队,每次让队首较矮者出列。i 扫左半、j 扫右半;某半耗尽,另一半剩余整体搬入。

示例 7-4 归并排序(切片版:递归新建子列表)

```
def merge(left, right):
    i = j = 0
    res = []
    while i < len(left) and j < len(right):
        if right[j] < left[i]:           # 右半更小才取右
            res.append(right[j])        # 相等取左 → 稳定
            j += 1
        else:
            res.append(left[i])
            i += 1
    return res + left[i:] + right[j:]   # 剩余半区整体搬入

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    left = merge_sort(a[:m])           # 切片:创建子列表副本
    right = merge_sort(a[m:])
    return merge(left, right)
```

实现建议:切片即复制,注意空间常数

Python 的切片 `a[:m]` 每次递归都新建一份子列表,总分配 $O(n \log n)$ 个元素,常数偏大;要省空间可用"下标区间 + 辅助列表"重写,但代码长一倍。生产代码直接 `sorted(a)`:Timsort 按段归并, $O(n)$ 空间。手写归并是为了理解分治,不是替代标准库。

7.4.3 稳定性与空间

归并排序**稳定**:合并"相等取左半"(严格小于才取右)。代价是 $O(n)$ 额外空间。工程价值:Python 的 `sorted` 用的 Timsort 就是"归并 + 插入"混合,检测到已有序段直接拼接,近有序数据近乎 $O(n)$ 。递归深度仅 $\log_2 n$,Python 默认递归上限 1000 也够用。

误区 4:merge 的边界错误(漏掉剩余半区或忘 return)

两个高频错:① 主循环后忘了把某半剩余元素拼进结果(奇数长度必错);② `merge` 忘了 `return` 返回 None——注意归并产生"新列表"必须传回来,不像插入排序那样原地改。调试技巧:用 5 个元素手工走一遍。

§7.5 快速排序:工程之王

7.5.1 思想与最坏情形

快排也是分治,但与归并的"先递归后合并"相反:难点在"分割"——选一个**枢轴(pivot)**,把数组分成"小于枢轴"与"大于枢轴"两半,枢轴落到最终位置,再递归排序两半。分割后无需合并——这是它常数小的根源。平均枢轴居中,递推式仍是

$T(n) = 2T(n/2) + n, O(n \log n)$; 最坏 $O(n^2)$: 每次分割极度不平衡——典型场景“已有序 + 首元素枢轴”, 退化为 $T(n) = T(n-1) + n$ 。

7.5.2 三数中值分割法

对付最坏情形的标准武器是**三数中值分割法(median-of-three)**: 取左端、中间、右端三个元素, 中值当枢轴, 顺便把三者排好序——左、右端变成“哨兵”, 枢轴藏中间。已有序数组会选中真正的中间值, 分割永远均衡, 退化被根治。

示例 7-5 快速排序: 三数中值 + 分割 + 小区间插入

```
CUTOFF = 16    # 长度 ≤ 16 改用插入排序

def median3(a, left, right):
    center = (left + right) // 2
    if a[center] < a[left]:
        a[left], a[center] = a[center], a[left]
    if a[right] < a[left]:
        a[left], a[right] = a[right], a[left]
    if a[right] < a[center]:
        a[center], a[right] = a[right], a[center]
    a[center], a[right - 1] = a[right - 1], a[center] # 枢轴藏到 right-1
    return a[right - 1]

def quick_sort(a, left=0, right=None):
    if right is None:
        right = len(a) - 1
    if left + CUTOFF <= right:
        pivot = median3(a, left, right)
        i, j = left, right - 1
        while True:
            i += 1
            while a[i] < pivot:          # 找 ≥ 枢轴者(a[left] 哨兵)
                i += 1
            j -= 1
            while pivot < a[j]:          # 找 ≤ 枢轴者(a[right-1] 哨兵)
                j -= 1
            if i < j:
                a[i], a[j] = a[j], a[i]
            else:
                break
        a[i], a[right - 1] = a[right - 1], a[i] # 枢轴归位
        quick_sort(a, left, i - 1)
        quick_sort(a, i + 1, right)
    else:
        insertion_sort(a, left, right)    # 小区间插入($7.1)
```

为什么循环不会越界? 三数中值已保证 $a[\text{left}] \leq \text{枢轴}$ 且 $a[\text{right}] \geq \text{枢轴}$, 两个“哨兵”让 $i += 1$ 与 $j -= 1$ 必然停下——这正是“顺便排好序”的深意。

7.5.3 小区间插入排序优化

快排递归到小区间时,插入排序反而更快:递归有开销、插入常数极小、缓存友好。工程实现普遍在长度 ≤ 16 时切换: `CUTOFF = 16 = "快排大区 + 插入小区"`。Timsort 与 `std::sort` 也这么做。

7.5.4 ★从 C 的 qsort 到 Python 的 sorted

C 的 qsort(函数指针 + void*)	Python 的 sorted(key=)(Timsort)
<code>qsort(a, n, sizeof(int), cmp):</code> 返回负/零/正	<code>sorted(a):</code> 默认升序,一行,默认稳定
<code>int cmp(const void*, const void*)</code> 手动强转,无类型检查	<code>key=lambda x: x</code> 提取排序键,元素类型任意
自定义顺序只能写死返回值规则	<code>key=lambda x: -x</code> 降序; <code>cmp_to_key</code> 桥接三值比较

示例 7-6 sorted 的四种用法

```
from functools import cmp_to_key
v = [5, 2, 8, 1, 9]
sorted(v) # 升序(返回新列表)
sorted(v, reverse=True) # 降序
sorted(v, key=lambda x: x % 10) # 按个位数升序
names = ["bob", "alice", "carl"]
sorted(names, key=len) # 按长度升序
sorted(names, key=cmp_to_key(lambda x, y: x - y)) # 三值比较桥接,语义同 qsort
```

⚠ 误区 5:把 C 的三值比较函数直接当 key 用

`qsort` 的比较函数收两个参数、返回"负/零/正"; `sorted` 的 `key=` 只收一个参数、返回排序键。把 C 的 `cmp` 直接塞进 `key=` 立刻 `TypeError`。正确姿势: `sorted(a, key=cmp_to_key(cmp))`。这是 C 迁移时最高频的报错。

7.5.5 快排的稳定性

快排**不稳定**:分割的"交换"会把相等元素隔空换位。反例:[3, 5_a, 2, 5_b, 1],取 3 当枢轴:分割时 5_a 与 1 交换,越过 5_b,次序破坏。需要稳定:小数据用插入,大数据直接用 `sorted`——它本来就是稳定的。

§7.6 桶式排序与外部排序:跳出"比较"的框架

7.6.1 桶式排序:用空间换时间

前面五种排序都属于"比较排序"。桶式排序(bucket sort)完全不做比较:把值直接当下标,数每个值出现几次,再按顺序倒出来——即计数排序(counting sort)(每桶一个值)。复杂度 $O(n + M)$, M 是值域; $M = O(n)$ 时即 $O(n)$ 。

示例 7-7 计数排序(前缀和 + 倒序,稳定)


```
def counting_sort(a, M):
    cnt = [0] * M
    out = [0] * len(a)
    for x in a:
        cnt[x] += 1
    for i in range(1, M):
        cnt[i] += cnt[i - 1]
    for x in reversed(a):
        cnt[x] -= 1
        out[cnt[x]] = x
    return out
```

值域 $[0, M)$ 的非负整数
cnt 兼作位置账本
1. 计数
2. 前缀和
3. 倒序扫描
稳定: 相等后到先取

C 的计数排序

```
int *cnt = calloc(M,
sizeof(int)); 用完 free, 忘了即泄漏
```

Python 的计数排序

```
cnt = [0] * M 自动管理内存
```

7.6.2 比较下界: $\Omega(n \log n)$ 的决策树直觉

为什么桶排能是 $O(n)$? 因为任何只靠"比较"的排序都有下界 $\Omega(n \log n)$, 而桶排不比较。一句话直觉: 一次比较最多把 $n!$ 种排列一分为二; 要确定唯一正确排列, 至少需要 $\log_2(n!) \approx n \log_2 n$ 次比较。 $O(n)$ 排序必须另辟蹊径——利用值域结构, 而非元素间比较。

桶排的适用条件因此苛刻: ① 数据是(或可映射成)小范围整数; ② 值域与数据量同阶; ③ 分布大致均匀。工程上主要用于成绩统计、颜色直方图、基数排序底层($O(d(n+M))$)。

7.6.3 外部排序: 内存装不下时

外部排序(external sort) 解决"数据比内存大"的问题(数据库、日志)。思路是归并的外延: 数据切成若干块, **每块内存排序后写回磁盘**; 再开小顶堆, **放每个已排块的当前最小元素, 每次弹出全局最小者写入输出, 再从该块补进下一个**——弹出、补入、再弹出直至读完; 输出仍装不下就多趟合并。分块排序用 §7.4 的归并, "多路取最小"正是第6章的小顶堆——**外部排序 = 归并思想 + 堆结构 + 磁盘代替内存**。

⚠ 误区 6: 对不满足条件的数据用桶排

对 10^6 个浮点数或值域 10^9 的整数用计数排序, 内存直接爆炸(要 M 个计数器)。判断口诀: 值域多大? 是整数吗? 分布均匀吗? 任何一条不满足, 老老实实用 $O(n \log n)$ 通用排序。工程上 99% 的场景都是通用排序, 桶排是值域受限场景的特供品。

§7.7 排序算法总表:选型决策

7.7.1 七种排序全景表

算法	平均	最坏	空间	稳定性
插入排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
希尔排序	约 $O(n^{3/2})$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n \log n)$	$O(n \log n)$	$O(1)$	不稳定
归并排序	$O(n \log n)$	$O(n \log n)$	$O(n)$	稳定
快速排序	$O(n \log n)$	$O(n^2)$	$O(\log n)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(1)$	稳定
桶排(计数)	$O(n + M)$	$O(n + M)$	$O(M)$	稳定

读表要点: 只有插入、冒泡能利用已有序(最好 $O(n)$); 只有归并和桶排稳定; 只有堆排最坏 $O(n \log n)$ 且 $O(1)$ 空间; 快排平均最快但最坏 $O(n^2)$ 。没有一种全占——选型即权衡。

7.7.2 选型决策树

- 数据量很小或基本有序 → **插入排序**
- 需要稳定、空间允许 → **归并排序**(Python 的 `sorted` 即稳定)
- 内存紧张、最坏必须保证 → **堆排序**
- 通用场景、要快 → **快速排序**
- 值域小的整数 → **桶排/计数排序**
- 其余一切情况 → **标准库 `sorted/sort`**, 不要手写

7.7.3 标准库是工程默认

最后一条最重要: 工程上永远优先 `sorted(a) / a.sort()` ——它是 Timsort: 归并为主, 检测到已有序段直接拼接(近有序近乎 $O(n)$), 小区间自动切插入排序, 而且默认稳定。本章手写的优化标准库全部内置, 还多一道稳定保证; C++ 的 `std::sort` 是 introsort(快排为主 + 防退化堆排), 语义相同。手写排序的意义: 理解原理、应对考试; 生产代码直接用标准库——本章实验"手写三种排序并统计比较次数", 正是为亲手验证第2章的复杂度理论。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
插入排序	随机/逆序数组	$O(n^2)$	$O(n^2)$
插入排序	已有序数组	$O(n)$	$O(n)$
冒泡排序	相邻交换	$O(n^2)$	$O(n^2)$
希尔排序	希尔增量	约 $O(n^{3/2})$	$O(n^2)$
希尔排序	Hibbard 增量	约 $O(n^{3/2})$	$O(n^{3/2})$
堆排序	建堆阶段	$O(n)$	$O(n)$
堆排序	完整排序	$O(n \log n)$	$O(n \log n)$
归并排序	完整排序	$O(n \log n)$	$O(n \log n)$
归并排序	额外空间	$O(n)$	$O(n)$
快速排序	随机数据	$O(n \log n)$	$O(n \log n)$
快速排序	已有序 + 首元素枢轴	$O(n^2)$	$O(n^2)$
桶排(计数排序)	值域 M	$O(n + M)$	$O(n + M)$
比较下界	基于比较的排序	$\Omega(n \log n)$ (决策树)	
归并两个有序数组	线性合并	$O(n)$	$O(n)$
外部排序	多路归并	磁盘 I/O 主导;比较部分 $O(n \log n)$	

最小必会清单

- 能默写插入排序(key 暂存三步),用 Σ 推导最坏 $O(n^2)$ 、近有序 $O(n)$
- 能说出希尔排序动机、希尔增量与 Hibbard 差别、复杂度约 $O(n^{3/2})$
- 能默写原地堆排序(0 基下滤、建堆、交换缩小),说出它不稳定并给反例
- 能默写归并的 merge,说清"相等取左"保证稳定、切片复制的空间代价
- 能默写快排的三数中值 + 分割 + 小区间插入,解释哨兵防越界
- 能说出快排最坏 $O(n^2)$ 何时发生、三数中值为何能治
- 能说出比较下界 $\Omega(n \log n)$ 的决策树直觉与桶排为何突破
- 能说出桶排三个适用条件,写出稳定版计数排序
- 能背出七种排序复杂度总表(最好/平均/最坏/空间/稳定性)
- 能写出 `sorted` 的 `key=` 与 `cmp_to_key`,说出与 `qsort` 三值语义的区别

自测题

- Q** 1. 插入排序对已有序数组的复杂度是多少?为什么?

✅ 参考答案

$O(n)$ 。每个元素与前驱比较 1 次就停止,共 $n-1$ 次比较、0 次移动。这也是"近有序接近 $O(n)$ "的来源:总代价 = 逆序对数。

Q 2. 堆排序为什么不稳定?构造一个反例。

✅ 参考答案

建堆与"根与末尾交换"都可能让相等元素换位。反例:[$4_a, 4_b, 3$] 建最大堆后根 4_a 与末尾 3 交换得 [$3, 4_b, 4_a$], 4_b 最终在前。

Q 3. 归并排序的空间复杂度是多少?为什么它不能像堆排那样原地?

✅ 参考答案

$O(n)$ 辅助数组。合并时两半在原数组中交错,原地交换会破坏"两半各自有序"的前提;原地归并要么 $O(n^2)$ 时间,要么复杂块交换技巧,得不偿失。

Q 4. 快排的最坏情形是什么?三数中值分割法为什么能缓解?

✅ 参考答案

每次分割都极度不平衡时(递推式 $T(n) = T(n-1) + n \rightarrow O(n^2)$),典型场景是已有序数组配"首元素枢轴"。三数中值取左、中、右的中值:已有序数组的中值恰是中间值,分割均衡;顺带排好 $a[\text{left}] \leq \text{枢轴} \leq a[\text{right}]$ 作哨兵。

Q 5. 为什么桶排能做到 $O(n)$,却不受"比较下界 $\Omega(n \log n)$ "的限制?

✅ 参考答案

决策树论证只适用于"通过比较获得信息"的算法:每次比较最多把 $n!$ 种排列一分为二,故至少 $\log_2(n!) \approx n \log_2 n$ 次。桶排不比较,直接用值域做索引(计数),绕开决策树,所以可 $O(n)$;代价是空间 $O(M)$ 与苛刻的适用条件。

章末实验题:三种排序的性能对比实验

实验 7:三种排序性能对比(三种数据分布)

实验目标:实现插入、归并、快排,在随机/近有序/逆序三种分布上实测 $n = 1000 \dots 64000$ 的耗时,用" n 翻倍时间比值"验证理论复杂度。

实验要求:

- 任务 1:实现 `insertion_sort`、`merge_sort`、`quick_sort`(\$7.1/\$7.4/\$7.5)
- 任务 2:三种分布 $\times$ 七个规模 n 的耗时表;插入 $n > 8000$ 标注"(推算)"
- 任务 3:与 `sorted` 对比验证输出一致,含重复元素
- 任务 4:" n 翻倍时间比值"与理论对比:插入 $\times 4(n^2)$ 、归并/快排 $\approx \times 2(n \log n)$
- 任务 5(加分):统计比较/交换次数,验证插入比较次数 $\approx n^2/4$ (逆序对)

实验环境:Python 3.8+。运行: `python lab7.py`。

第一步 排务必用三数中值 + CUTOFF=16 小区间插入(\$7.5 完整版),否则逆序数据退化 $O(n^2)$,与理论演示矛盾

第二步 时取"同数据 3 次最小值"、用副本,固定种子 12345 可复现;近有序用"已排序 + 固定 20 次随机交换"生成(逆序对 $O(n)$),才能观察插入的 $O(n)$ 特性

第三步 务 4 比值只看"翻倍相邻行",小 n 计时噪声大,偏离属正常,看趋势即可

✅ 参考答案(完整可运行程序,分六段)

```
# lab7.py — 三种排序的性能对比实验(插入/归并/快排)
# 运行: python lab7.py
import random
import time

random.seed(12345)      # 固定种子,输出可复现

CUTOFF = 16             # 快排小区间改用插入排序的阈值($7.5)

# ----- 任务 5(可选加分):比较/移动计数器 -----
COUNT = False          # 置 True 时统计,性能测试期间保持 False
cmp_cnt = 0
move_cnt = 0

def lt(x, y):
    global cmp_cnt
    if COUNT:
        cmp_cnt += 1
    return x < y

# ----- 任务 1:插入排序($7.1) -----
def insertion_sort(a, left=0, right=None):
    if right is None:
        right = len(a) - 1
    for i in range(left + 1, right + 1):
        key = a[i]
        j = i - 1
        while j >= left and lt(key, a[j]): # 严格大于才右移:相等不动,稳定
            a[j + 1] = a[j]               # 元素右移
            j -= 1
        global move_cnt
        if COUNT:
            move_cnt += 1
        a[j + 1] = key                    # 落位
        if COUNT:
            move_cnt += 1
```

lab7.py(第二段:归并排序)

```

# ----- 任务 1:归并排序($7.4,切片 + 合并) -----
def merge(left, right):
    global move_cnt
    i = j = 0
    res = []
    while i < len(left) and j < len(right):
        if lt(right[j], left[i]):          # 右半更小才取右:相等取左,稳定
            res.append(right[j])
            j += 1
        else:
            res.append(left[i])
            i += 1
    if COUNT:
        move_cnt += 1
    res += left[i:] + right[j:]          # 剩余部分直接拼接
    return res

def merge_sort(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    left = merge_sort(a[:m])
    right = merge_sort(a[m:])
    return merge(left, right)          # 切片:创建子列表副本($7.4)

```

lab7.py(第三段:快速排序与数据生成)

```

# ----- 任务 1:快速排序($7.5,三数中值 + 小区间插入) -----
def median3(a, left, right):
    center = (left + right) // 2
    if lt(a[center], a[left]):
        a[left], a[center] = a[center], a[left]
    if lt(a[right], a[left]):
        a[left], a[right] = a[right], a[left]
    if lt(a[right], a[center]):
        a[center], a[right] = a[right], a[center]
    a[center], a[right - 1] = a[right - 1], a[center] # 枢轴藏到 right-1
    return a[right - 1]

def quick_sort(a, left=0, right=None):
    if right is None:
        right = len(a) - 1
    if left + CUTOFF <= right:
        pivot = median3(a, left, right)
        i, j = left, right - 1
        while True:
            i += 1
            while lt(a[i], pivot): # 哨兵 a[left]<=pivot 保证能停
                i += 1
            j -= 1
            while lt(pivot, a[j]): # a[right-1]==pivot 保证能停
                j -= 1
            if i < j:
                a[i], a[j] = a[j], a[i]
            else:
                break
        a[i], a[right - 1] = a[right - 1], a[i] # 枢轴归位
        quick_sort(a, left, i - 1)
        quick_sort(a, i + 1, right)
    else:
        insertion_sort(a, left, right) # 小区间优化($7.1 预告)

# ----- 任务 2:三种数据分布 -----
def make_data(n, kind):
    if kind == "随机":
        a = list(range(n))
        random.shuffle(a)
    elif kind == "近有序":
        a = list(range(n))
        for _ in range(20): # 固定 20 次随机交换:逆序对 O(n)
            x, y = random.randrange(n), random.randrange(n)
            a[x], a[y] = a[y], a[x]
    else: # 逆序
        a = list(range(n, 0, -1))
    return a

```

lab7.py(第四段:计时)


```
def ms_time(func, a, copies=True):
    """计时:同数据重复 3 次取最小值,压制系统噪声;copies=True 时每次用副本。"""
    best = float("inf")
    for _ in range(3):
        b = a[:] if copies else a
        t0 = time.perf_counter()
        func(b)
        best = min(best, (time.perf_counter() - t0) * 1000)
    return best
```

lab7.py(第五段:正确性验证与耗时表)

```
def main():
    sizes = (1000, 2000, 4000, 8000, 16000, 32000, 64000)
    kinds = ("随机", "近有序", "逆序")
    times = {k: {"插入": {}, "归并": {}, "快排": {}} for k in kinds}

    # 任务 3:正确性验证(与 sorted 输出一致,含重复元素)
    ok = True
    for kind in kinds:
        a = make_data(500, kind)
        std = sorted(a)
        b = a[:]; insertion_sort(b); ok &= (b == std)
        ok &= (merge_sort(a[:]) == std)
        b = a[:]; quick_sort(b); ok &= (b == std)
    a = [random.randint(0, 20) for _ in range(500)] # 重复元素
    std = sorted(a)
    b = a[:]; insertion_sort(b); ok &= (b == std)
    ok &= (merge_sort(a[:]) == std)
    b = a[:]; quick_sort(b); ok &= (b == std)
    print(f"任务 3:三种算法与 sorted 输出一致: {ok}")

    # 任务 2:性能表(插入 n>8000 按 x4 规律推算,避免无谓等待)
    print("任务 2:耗时表(单位 ms;插入排序 n>8000 为(推算))")
    for kind in kinds:
        print(f"\n[{kind}]")
        print("n      插入      归并      快排")
        for n in sizes:
            a = make_data(n, kind)
            row = [f"{n}"]
            if n <= 8000:
                t = ms_time(insertion_sort, a[:])
                times[kind]["插入"][n] = t
                row.append(f"{t:9.1f}")
            else:
                row.append(" (推算)")
            t = ms_time(merge_sort, a[:], copies=False) # 归并不改动原列表
            times[kind]["归并"][n] = t
            row.append(f"{t:9.1f}")
            t = ms_time(quick_sort, a[:])
            times[kind]["快排"][n] = t
            row.append(f"{t:9.1f}")
        print(" ".join(row))
```

lab7.py(第六段:比值分析、统计与结论)

```

# 任务 4:比值分析(理论:插入  $\times 4$ ;归并/快排  $\approx \times 2$ )
print("\n任务 4:n 翻倍时实测时间比值(理论: $n^2$  算法  $\times 4$ , $n \log n$  算法  $\approx \times 2$ )")
theory = {"插入": " $n^2 \rightarrow \times 4$ ", "归并": " $n \log n \rightarrow \approx \times 2$ ", "快排": " $n \log n \rightarrow \approx \times 2$ "}
for alg in ("插入", "归并", "快排"):
    line = f"{alg}({theory[alg]}): "
    for kind in kinds:
        ts = sorted(times[kind][alg].items())
        rs = [f"{ts[i+1][1]/ts[i][1]:.1f}" for i in range(len(ts) - 1)]
        line += f"{kind}[' '.join(rs)] "
    print(line)

# 任务 5(加分):比较/移动次数统计
global COUNT, cmp_cnt, move_cnt
COUNT = True
a = make_data(1000, "随机")
cmp_cnt = move_cnt = 0
insertion_sort(a[:])
print(f"\n任务 5(n=1000 随机):插入 比较={cmp_cnt} 移动={move_cnt}")
cmp_cnt = move_cnt = 0
merge_sort(a[:])
print(f"          归并 比较={cmp_cnt} 移动={move_cnt}(拼接不计)")
cmp_cnt = move_cnt = 0
quick_sort(a[:])
print(f"          快排 比较={cmp_cnt} 交换={move_cnt}")
COUNT = False

print("\n结论:实测比值与理论吻合—插入排序 n 翻倍时间约  $\times 4$ ,"
      "\n      归并/快排约  $\times 2$ ;逆序数据上插入排序退化最明显,"
      "\n      近有序数据上插入排序接近  $O(n)$ ,快排三数中值使逆序数据不再退化。")

if __name__ == "__main__":
    main()

```

示例输出(本机真实运行,不同机器数值不同,比值规律一致):

任务 3:三种算法与 sorted 输出一致: True
任务 2:耗时表(单位 ms;插入排序 $n > 8000$ 为(推算))

[随机]

n	插入	归并	快排
1000	13.8	1.0	0.6
2000	61.4	2.2	1.3
4000	257.4	4.8	2.8
8000	1019.2	10.8	6.3
16000	(推算)	22.8	12.8
32000	(推算)	50.4	28.5
64000	(推算)	117.8	66.5

[近有序] 表与 [逆序] 表见完整输出:插入在近有序上快两个数量级(8000 时仅 5.4ms),逆序上最慢(2025.0ms)

任务 4:n 翻倍时实测时间比值(理论: n^2 算法 $\times 4$, $n \log n$ 算法 $\approx \times 2$)
插入($n^2 \rightarrow \times 4$): 随机[4.5 4.2 4.0] 近有序[2.3 1.8 1.5] 逆序[4.2 4.2 4.0]
归并($n \log n \rightarrow \approx \times 2$): 随机[2.1 2.2 2.3 2.1 2.2 2.3](近有序/逆序各列也均 ≈ 2)
快排($n \log n \rightarrow \approx \times 2$): 随机[2.2 2.2 2.2 2.0 2.2 2.3](同上)

任务 5($n=1000$ 随机):插入 比较=246311 移动=246323
归并 比较=8720 移动=8720(拼接不计)
快排 比较=10882 交换=3088

读表要点:① 插入在随机/逆序上比值 $\approx \times 4$ (吻合 n^2),近有序上 $\approx \times 2$ (吻合 $O(n)$);② 归并/快排 $\approx \times 2$ (吻合 $n \log n$, 三种分布皆然);③ 快排逆序不退化——三数中值选中中间值,§7.5 优化的实战验证;④ 插入比较 246311 $\approx 1000^2/4$,落在平均情形的理论上。

设计说明:① `lt` 计数钩子使比较/移动定义与正文一致;② 计时取 3 次最小值、用副本(归并本来就返回新列表,直接传原件);③ 插入 $n > 8000$ 标"(推算)": $\times 4$ 规律已由 1000→8000 确认,与第2章实验一脉相承;④ 近有序用固定 20 次交换生成(逆序对 $O(n)$);⑤ 固定种子 12345,输出可复现。

下一章预告:第8章 不相交集——union/find 与路径压缩,一种"只回答是/否连通"的轻量数据结构;第9章图论算法中的 Kruskal 最小生成树将直接使用它。